EconoCausal: Repo Review and Feature Plan

Review of github.com/Krishna8208863439/EconoCausal, prepared 4 November 2026

Scope note: This review is based on the README and `PROJECT_AUDIT_REPORT.md` (audit dated 15 Oct 2026). GitHub blocked automated access to the source folders (`ml/`, `backend/`, `frontend/`), so the code itself was not read. Anything about implementation is inferred from those two documents and should be checked against the code.

What the repo is

EconoCausal is a causal-ML platform for discount targeting:

- **Data:** about 2,200 synthetic retail customers with a known true effect (+12.02 pp).
- **ML:** EconML `CausalForestDML` and `LinearDML` with 5-fold cross-fitting, plus a DoWhy DAG.
- **Evaluation:** uplift and Qini curves, and a 4-policy comparison (causal, blanket, random, no discount).
- **Optimizer:** a SciPy MILP that allocates a fixed budget.
- **Monitoring:** a KS, Chi² and PSI drift detector.
- **App:** a FastAPI backend with about 21 endpoints and an 8-page React/Plotly dashboard.

Fix these first

New features will sit on shaky ground until these are sorted.

1. **The headline result doesn't match the truth.** The model estimates +19.47 pp (CI roughly 5.7 to 33.3) against a true +12.02 pp, yet the README says confounding bias was removed. The treatment model's ROC-AUC is only 0.536, so assignment looks almost random. If so, the naive estimate (+20.86) shouldn't be 9 pp above the truth either. Something in the data generation, the estimator setup, or how "ground truth" is defined is off. This deserves investigation before anything else.
2. **The README contradicts itself.**
   - Test counts appear as 153, 99, 63 and 33.
   - Dataset size appears as 2,200 and 2,244 rows.
   - Charting is described as Plotly in one place and Recharts in another.

    - Section 30's benchmarks (RMSE 0.08, R^2 0.76, 18% improvement, `synthetic_customer_data.csv`) don't match Section 13 (7.3% lift).
3. **Several modules the README mentions aren't in the audit's file inventory.** These are `sensitivity_analysis.py`, `policy_evaluator.py`, `benchmarks.py` and the Dockerfile. The audit also says Docker is not configured. Either build them or remove the claims.
4. **Tests.** The Oct 15 audit found 120 of 153 tests blocked by a `gmpy2` crash on Python 3.14, but the README now says all pass. Pin Python 3.11 or 3.12, add GitHub Actions CI, and confirm the real count.
5. **Small items from the audit:** the currency default is GBP vs INR, CORS is `*`, the `.joblib` model and allocation CSV are gitignored, and commit history is thin.

Feature plan

Phase 1: make the economics real (highest value)

Feature	Why
Profit-aware objective (margin, discount cost, revenue outcome)	The optimizer currently targets purchase uplift, which isn't profit. The README itself warns about this.
Multi-level discounts (0/10/20%) with the optimizer choosing a level per customer	Addresses the "static discount" limitation. A multiple-choice knapsack.
Price elasticity / continuous treatment	The project is titled "Dynamic Pricing" but only does discount targeting. Per-segment elasticity and an optimal-price rule is the real pricing feature.
Uncertainty-aware allocation	Skip customers whose CI includes zero, or rank by the lower bound. The CIs are already computed.
"Do not disturb" flag	Negative-uplift customers (sleeping dogs) shown explicitly.

Phase 2: stronger causal validation

- DoWhy refutations (placebo, random common cause, data subset) and an E-value or Cinelli-Hazlett sensitivity analysis.
- An estimator bake-off (DRLearner, X/T/S-learners, causal forest, LinearDML) chosen on held-out Qini or R-loss.
- Doubly robust off-policy evaluation to compare policies properly.
- Validation on a real randomized dataset, such as Criteo Uplift or Hillstrom Email. Everything so far is synthetic, so this is the strongest credibility upgrade.

Phase 3: analyst-facing features

- A budget what-if slider with a marginal ROI curve and a suggested optimal budget.
- Explainability: SHAP or EconML's tree interpreter showing why a segment responds.
- A fairness audit of allocations by age group and other segments.
- An A/B test planner (sample size and power, holdout generation), plus result import to compare against the DML estimates.
- PDF export of reports.

Phase 4: engineering

- Docker Compose, CI/CD, and a model registry with versioning.
- Auto-retrain on drift alerts, and a database (Postgres or SQLite) in place of CSV and in-memory state.
- JWT auth and a restricted CORS policy.

Suggested order

1. Investigate the ATE discrepancy, then fix the tests and the README.
2. Profit-aware objective plus uncertainty-aware allocation (small effort, big credibility gain).
3. Multi-level discounts.
4. Refutations and off-policy evaluation.
5. A real randomized dataset.
6. Elasticity and the what-if simulator.

If the goal is an internship review or viva, put validation and fix-ups first. If the goal is a portfolio or product piece, put the real dataset and elasticity first.